Group Chat with Rooms + Presence

Pixel Panthers- Adrian Lopez, Isabel Villarreal, Emma Whitehead, Anthony Whitmore, Mallory Sorola

Overview

Introduction

Our goal was to create a multi-client chat service with rooms, presence updates, and messaging. To do so:

- We built a custom reliable transport protocol over UDP
- Implemented order delivery, ACKs, retransmissions, flow control, and checksums
- Handled packet loss, delay, duplication, and reordering
- Collected performance metrics
- Ensured program could perform under lossy network tests

Transport Protocol Design

Header Structure

- Version
- Flags
- ConnID (Connection ID)
- Seq/Ack numbers
- Payload Length
- Checksum

Selective Repeat for Reliability

- Multiple in-flight packets are allowed
- Only lost packets are retransmitted
- Out-of-order packets are saved (buffered) until the missing packets arrive
- Delivered in order

Time Out and Retransmission

- Each outgoing packet has its own timer
- Only a packet that times out is resent
- Unaffected packets continue as normal
- When retransmitted the window does NOT reset

Transport Protocol Design 2

Flow Control

- The receiver advertises window size (wnd)
- Sender limits how many packets are in flight

Checksum

- All packets include a checksum that covers a header and payload.
- When checksum fails, bad packets are discarded and trigger a retransmission

Clean Application-Layer API

connect(addr)

- Establishes reliable connection

send_msg(b)

- Sends message as in-order packet stream

on_message(callback)

- Registers a function that runs when a message is received

close()

- Gracefully closes connection

Application Design

- Our application uses a simple, slash-prefixed command grammar to maintain simplicity and promote intuitive client interaction.

Presence Notifications

Presence updates occur when a client joins, leaves, or disconnects unexpectedly from a room.

Error Handling

Robust handling of errors such as:

- Unknown commands
- Bad arguments
- Invalid logins
- Missing rooms
- Unauthorized actions
- Unexpected disconnects

Concurrency Model

The server uses a multi-threaded design for one thread per connected client, shared data structures like rooms, and user tables, and properlocking to prevent race conditions.

Application Design - Commands

Our chat client has only a few core commands:

- /join ROOM
  - join a chatroom
- /leave ROOM
  - leave a chatroom
- /msg ROOM <text>
  - send a message to a chatroom
- /dm USER <text>
  - send private message to specific user
- /quit
  - disconnect

Chat_server.py

Chat_server.py is used to host the chat server.

- Listens on one port (default UDP 9000)
- Accepts connections from multiple clients
- Authenticates users (register/login)
- Hosts multiple chat rooms
- Broadcasts messages to everyone in the same room
- Maintains active user lists
- Saves credentials to disk (accounts.json)

Chat_transport.py

Chat_transport.py is built on UDP with modifications.

- Guarantees messages arrive and in the right order
- Creates connections between client and server
- Resends lost data automatically
- Delivers messages to chat_client.py
- Tracks performance metrics
- CRC32 is used to detect corruption

Chat_client.py

Chat_client.py is the actual chat client program.

- The user is required to register an account with a unique username and password
    - Once they've registered, they can login with those same credentials

Upon login the user gets access to more features:

- Join a room of their choice and chat
- Privately direct message (DM) other users
- View message history
- Leave the room and join another
- Quit

Conclusion

In conclusion, we:

- Built a full custom transport-layer protocol above UDP
- Implemented key TCP-like features in our project like:
  - Selective Repeat ARQ
  - Custom headers
  - Flow Control
  - Retransmissions
  - Message Oriented API
- All while having the system perform reliably across three lossy test profiles that we provided

Overall, this project strengthened our practical understanding of transport-layer networking and demonstrated how protocol design decisions shape application performance.